In [ ]:
```
PAWAR SAKSHI MAHENDRA
ROLL NO - 49
PRACTICAL NO - 06
```

In [3]:
```
import numpy as np

class NeuralNetwork:
    def __init__(self, input_size, hidden_size, output_size):
        # weights and bias initialization
        self.W1 = np.random.randn(input_size, hidden_size)
        self.b1 = np.zeros((1, hidden_size))
        self.W2 = np.random.randn(hidden_size, output_size)
        self.b2 = np.zeros((1, output_size))

    # activation function
    def sigmoid(self, x):
        return 1 / (1 + np.exp(-x))

    # derivative of sigmoid
    def sigmoid_derivative(self, x):
        return x * (1 - x)

    # forward propagation
    def forward_propagation(self, X):
        self.z1 = np.dot(X, self.W1) + self.b1
        self.a1 = self.sigmoid(self.z1)
        self.z2 = np.dot(self.a1, self.W2) + self.b2
        self.y_hat = self.sigmoid(self.z2)
        return self.y_hat

    # backward propagation
    def backward_propagation(self, X, y, y_hat, lr):
        error = y - y_hat
        delta2 = error * self.sigmoid_derivative(y_hat)

        a1_error = delta2.dot(self.W2.T)
        delta1 = a1_error * self.sigmoid_derivative(self.a1)

        # update weights and bias
        self.W2 += self.a1.T.dot(delta2) * lr
        self.b2 += np.sum(delta2, axis=0, keepdims=True) * lr
        self.W1 += X.T.dot(delta1) * lr
        self.b1 += np.sum(delta1, axis=0, keepdims=True) * lr

        return error

    # training function
    def train(self, X, y, epochs, lr):
        for i in range(epochs):
            y_hat = self.forward_propagation(X)
            error = self.backward_propagation(X, y, y_hat, lr)

            if i % 1000 == 0:
```

```
                print("Error at epoch", i, ":", np.mean(np.abs(error)))

    # prediction with threshold
    def predict(self, X):
        y_hat = self.forward_propagation(X)
        return (y_hat >= 0.5).astype(int)


# XOR dataset
X = np.array([[0, 0],
              [0, 1],
              [1, 0],
              [1, 1]])

y = np.array([[0],
              [1],
              [1],
              [0]])

# create neural network
nn = NeuralNetwork(input_size=2, hidden_size=4, output_size=1)

# train the model
nn.train(X, y, epochs=10000, lr=0.1)

# predictions
predictions = nn.predict(X)

print("\nFinal Predictions:")
print(predictions)
```

```
Error at epoch 0 : 0.5002150211733821
Error at epoch 1000 : 0.4965349510216907
Error at epoch 2000 : 0.4632301049927057
Error at epoch 3000 : 0.3464371644580758
Error at epoch 4000 : 0.19417295410758803
Error at epoch 5000 : 0.12184013931382655
Error at epoch 6000 : 0.09035744980254361
Error at epoch 7000 : 0.07320065738780997
Error at epoch 8000 : 0.062362452062974416
Error at epoch 9000 : 0.054843913660222836

Final Predictions:
[[0]
 [1]
 [1]
 [0]]
```

In [ ]: